



Exclusão Mútua **Distribuída** **Algoritmos Distribuídos**

Prof Priscila Marques Kai

Algoritmos Baseados em Votação (Quorum)

A ideia dos algoritmos baseados em votação é evitar que um processo precise consultar todos os demais processos do sistema para entrar na seção crítica.

Em vez disso, cada processo possui um subconjunto de processos, chamado **quórum** ou **grupo de votação**. Quando um processo deseja acessar a seção crítica, ele envia uma solicitação apenas aos processos pertencentes ao seu quórum.

Se receber a autorização de todos os processos desse grupo, ele pode entrar na seção crítica. Caso contrário, deve aguardar até que as permissões necessárias sejam concedidas.



Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)

A principal vantagem dessa abordagem é a redução do número de mensagens trocadas, pois a decisão não depende da consulta a todos os processos do sistema, mas apenas a um conjunto menor cuidadosamente escolhido.

Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)

Um dos algoritmos mais conhecidos dessa categoria é o algoritmo de Maekawa, que organiza os processos em grupos de votação cujo tamanho é da ordem de $\sqrt{N}$, reduzindo o número de mensagens necessárias para controlar o acesso à seção crítica em sistemas descentralizados.

Referência:

Maekawa, M. A $\sqrt{N}$ Algorithm for Mutual Exclusion in Decentralized Systems. ACM Transactions on Computer Systems, v. 3, n. 2, 1985.

Algoritmo de **Maekawa (1985)**:

Cada processo P_i possui um distrito S_i , formado por um subconjunto de processos do sistema.

Para entrar na seção crítica, P_i precisa obter autorização dos processos pertencentes ao seu distrito.

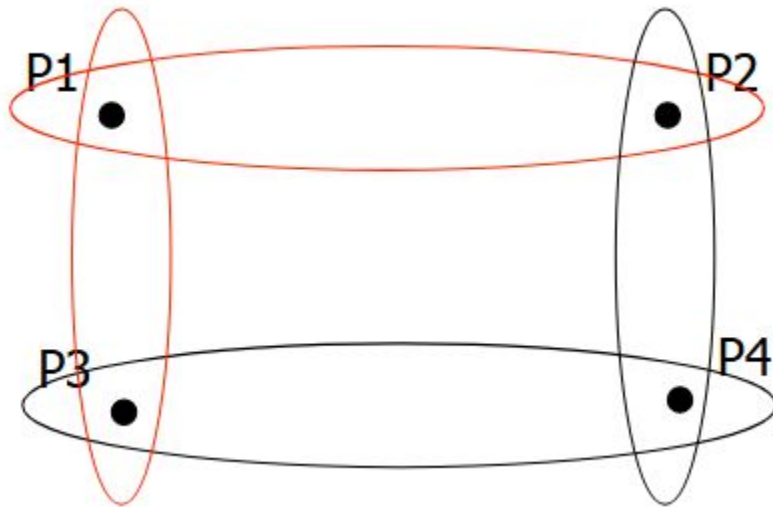
Esses distritos devem ser construídos de modo que:

- cada processo pertença ao seu próprio distrito;
- quaisquer dois distritos tenham pelo menos um processo em comum;
- os distritos tenham tamanhos semelhantes, quando possível;
- a participação dos processos nos distritos seja equilibrada;
- nenhum distrito seja completamente contido em outro.

Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)

A propriedade mais importante é a **interseção obrigatória**: dois processos diferentes sempre terão pelo menos um processo em comum em seus distritos. Como esse processo comum só pode conceder voto a um deles por vez, evita-se que dois processos entrem simultaneamente na seção crítica.

Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)



$S1 = \{p1, p2, p3\}$

$S2 = \{p1, p2, p4\}$

$S3 = \{p1, p3, p4\}$

$S4 = \{p2, p3, p4\}$

Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)

Uma forma prática de construir esses grupos é organizar os processos em uma matriz.

Por exemplo, se temos $N=9$ processos, podemos colocá-los em uma matriz 3×3 :

Para cada processo, seu grupo de votação será formado pelos processos da **mesma linha** e da **mesma coluna**.

P_1	P_2	P_3
P_4	P_5	P_6
P_7	P_8	P_9

Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)

Por exemplo, para P_5 , temos:

$$S_5 = \{P_2, P_4, P_5, P_6, P_8\}$$

P_1	P_2	P_3
P_4	P_5	P_6
P_7	P_8	P_9

Por exemplo, para $P_{5'}$ temos:

$$S_1 = \{P_1, P_2, P_3, P_4, P_7\}$$

$$S_2 = \{P_1, P_2, P_3, P_5, P_8\}$$

$$S_3 = \{P_1, P_2, P_3, P_6, P_9\}$$

$$S_4 = \{P_1, P_4, P_5, P_6, P_7\}$$

$$S_5 = \{P_2, P_4, P_5, P_6, P_8\}$$

$$S_6 = \{P_3, P_4, P_5, P_6, P_9\}$$

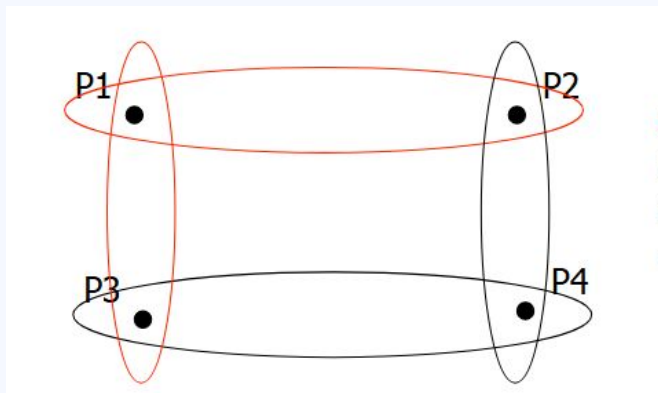
$$S_7 = \{P_1, P_4, P_7, P_8, P_9\}$$

$$S_8 = \{P_2, P_5, P_7, P_8, P_9\}$$

$$S_9 = \{P_3, P_6, P_7, P_8, P_9\}$$

P_1	P_2	P_3
P_4	P_5	P_6
P_7	P_8	P_9

Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)



$$\begin{aligned} S_1 &= \{P_1, P_2, P_3\} \\ S_2 &= \{P_1, P_2, P_4\} \\ S_3 &= \{P_1, P_3, P_4\} \\ S_4 &= \{P_2, P_3, P_4\} \end{aligned}$$

P_1	P_2
P_3	P_4

Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)

Uma vez definidos os distritos S_i e seus respectivos membros, a questão passa a ser:

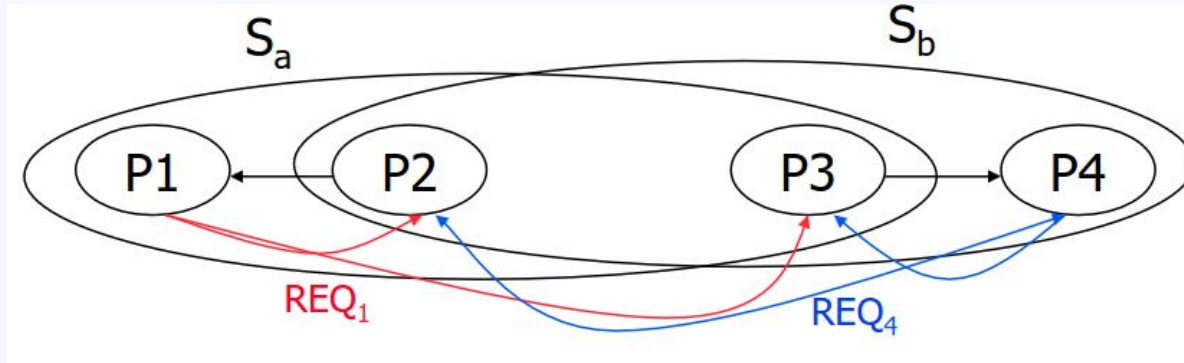
como usar esses distritos para garantir a exclusão mútua?

Uma primeira tentativa seria o seguinte algoritmo simples:

- Quando um processo P_i deseja entrar na seção crítica, ele envia uma mensagem **REQ** para todos os processos pertencentes ao seu distrito S_i .
- Cada processo que recebe a solicitação responde com **OK**, desde que ainda não tenha concedido seu voto a outro processo.
- O processo P_i só entra na seção crítica depois de receber **OK** de todos os membros de S_i .
- Ao sair da seção crítica, P_i envia uma mensagem **RELEASE** para todos os processos de S_i , liberando os votos recebidos.

O problema desse algoritmo ingênuo é a **possibilidade de deadlock**.

Situação de deadlock



Como decidir qual requisição veio primeiro?

Situação de deadlock

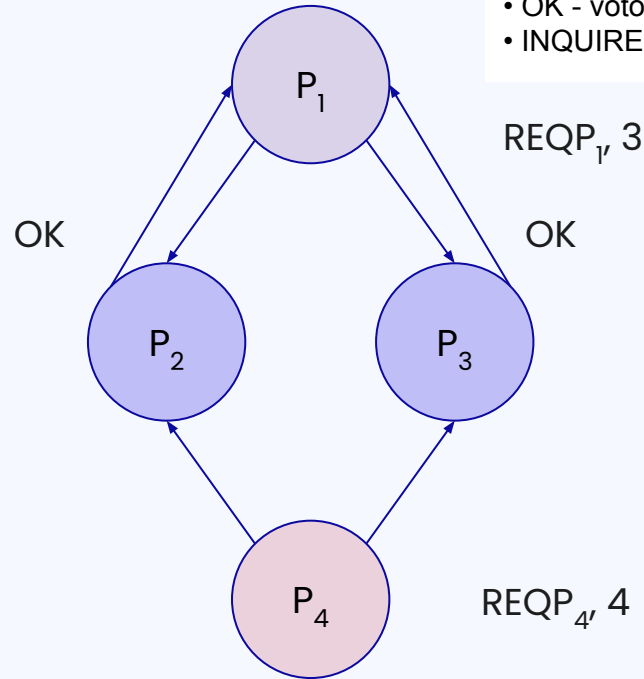
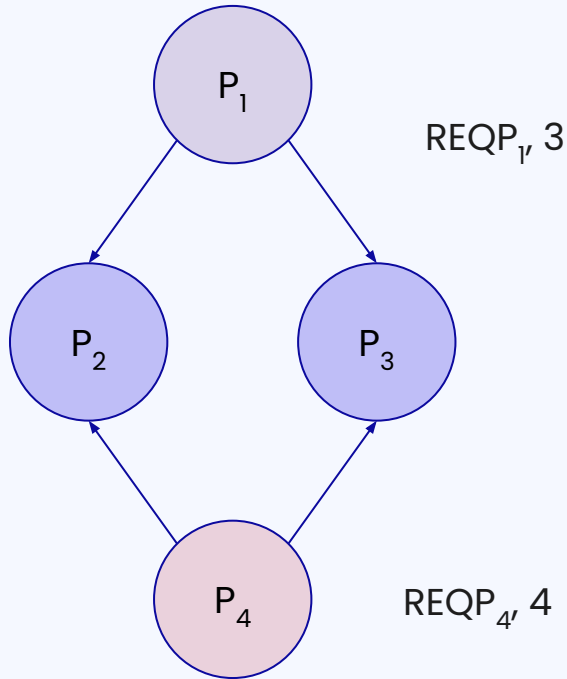
Uma forma de resolver o problema de deadlock em algoritmos de votação é utilizar **relógios lógicos de Lamport** para ordenar as requisições de entrada na seção crítica.

Solução de Sanders [Sanders, 1987; Sanders, 1996]:

- cada mensagem **REQ** é enviada junto com um **timestamp**, indicando o instante lógico em que o pedido foi feito.
- quando dois processos disputam votos de distritos que se intersectam, a requisição com menor timestamp tem prioridade.
- Se um processo já concedeu voto a uma requisição e depois recebe outra com timestamp menor, ele envia uma mensagem **INQUIRE** ao processo que recebeu o voto, solicitando sua devolução.

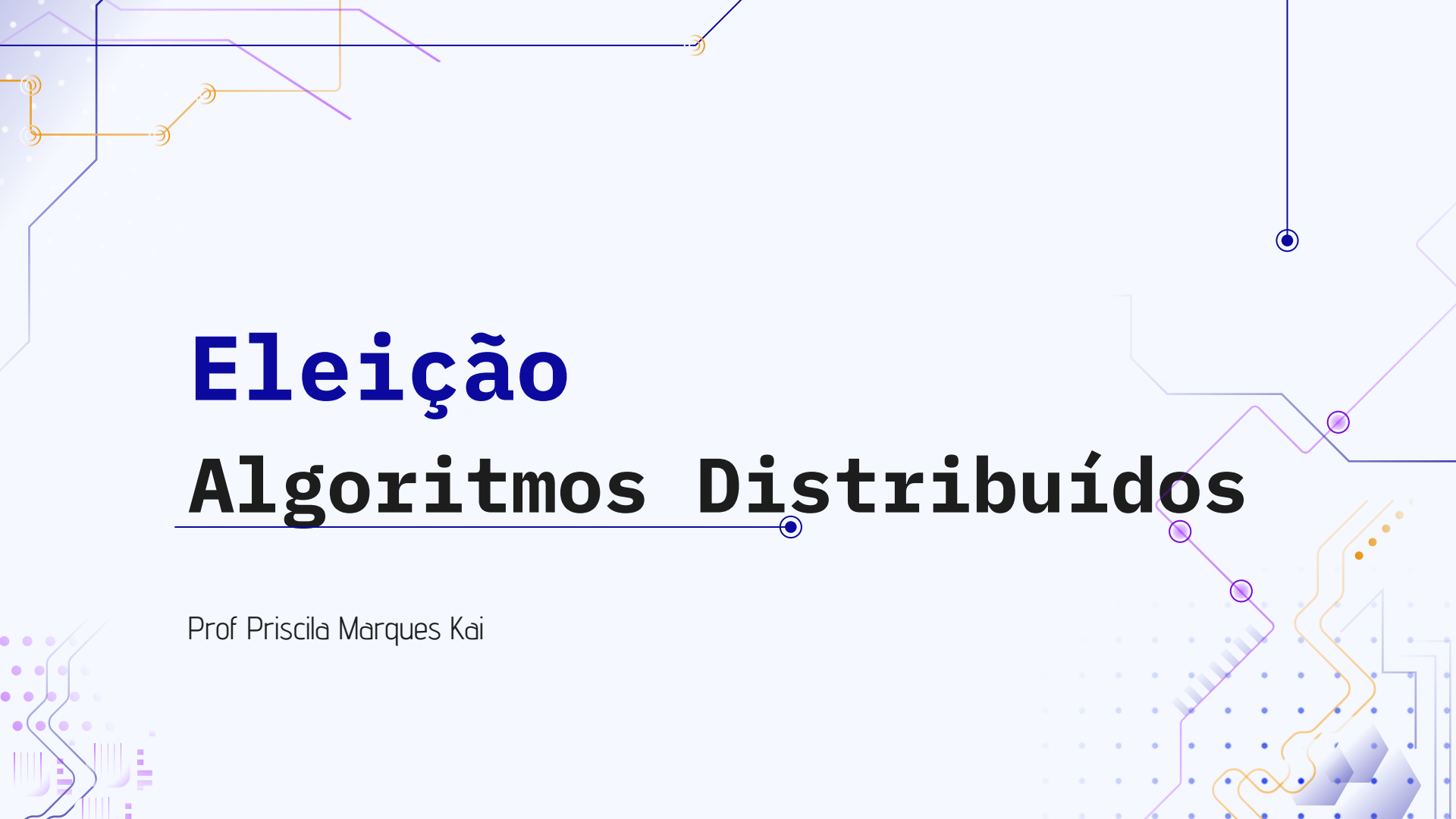
Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)

Situação de deadlock



Tipos de Mensagem:

- REQ - pedido de entrada na SC
- RELEASE - notificação de saída da SC
- RELINQUISH - devolução dos votos
- OK - voto para entrada na SC
- INQUIRE - solicitação de devolução do voto

The background features a light blue gradient with abstract circuit-like patterns. Purple and orange lines, some straight and some curved, crisscross the slide. Small circles, some solid and some hollow, are placed at various points along these lines, resembling nodes or components in a network diagram.

Eleição

Algoritmos Distribuídos

Prof Priscila Marques Kai

ALGORITMOS DE ELEIÇÃO

Muitos algoritmos distribuídos exigem que um processo atue como coordenador, iniciador ou possui algum outro papel especial.

Para isso, veremos algoritmos com a função de eleger um coordenador.

Se todos os processos forem iguais, sem características distintivas, não há como selecionar um deles para ser especial.

- assumiremos que cada processo P possui um identificador único ($\text{id}(P)$).

ALGORITMOS DE ELEIÇÃO

Algoritmos de eleição tentam localizar o processo com o identificador mais alto e designá-lo como coordenador.

- algoritmos diferem na maneira como localizam o coordenador.
- assumimos que cada processo conhece o identificador de todos os outros processos.
 - processos não sabem quais processos estão ativos e quais estão inativos.

ALGORITMOS DE ELEIÇÃO

Objetivo de um algoritmos de eleição: garantir que, quando uma eleição começar, ela termine com todos os processos concordando sobre quem será o novo coordenador.

Exemplos de algoritmos de Eleição:

- Algoritmo Bully
- Algoritmo Ring

ALGORITMO Bully (Valentão)

O algoritmo do bully, proposto por Garcia-Molina (1982) é um algoritmo para a eleição de um coordenador.

Considere um conjunto de N processos $\{P_0, P_1, \dots, P_{N-1}\}$, onde cada processo possui um identificador $id(P_k) = k$.

- Quando um processo detecta que o coordenador atual não está mais respondendo, ele inicia uma eleição.

ALGORITMO Bully

O procedimento seguido por um processo P_k é o seguinte:

1. P_k envia uma mensagem ELECTION para todos os processos com identificadores maiores — ou seja, para $P_{k+1}, P_{k+2}, \dots, P_{N-1}$.
2. Se nenhum desses processos responder, P_k vence a eleição e se torna o novo coordenador.
3. Se algum processo com identificador maior responder, esse processo assume a responsabilidade pela eleição, e P_k encerra sua participação.

ALGORITMO Bully

A qualquer momento, um processo pode receber uma mensagem ELECTION de outro processo com um identificador menor.

- o processo que recebe a mensagem envia uma mensagem OK de volta ao remetente, assumindo a responsabilidade pela eleição.
- E inicia sua própria eleição.

Eventualmente, todos os processos desistem, exceto um — e esse se torna o novo coordenador.

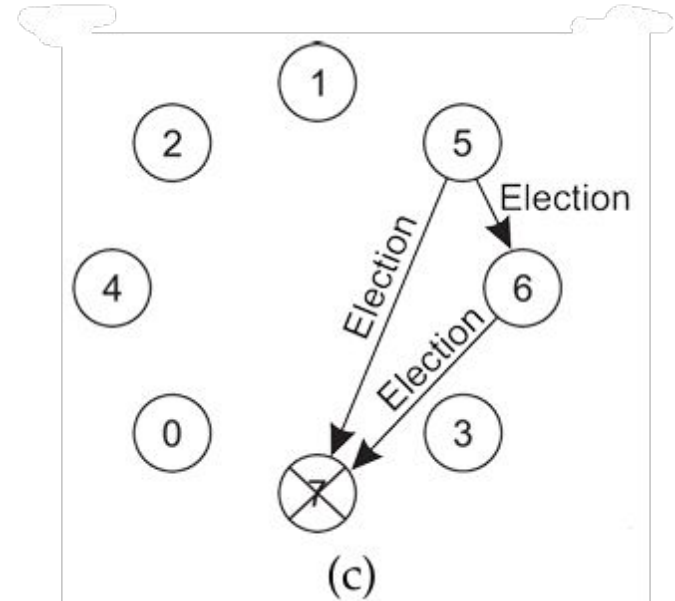
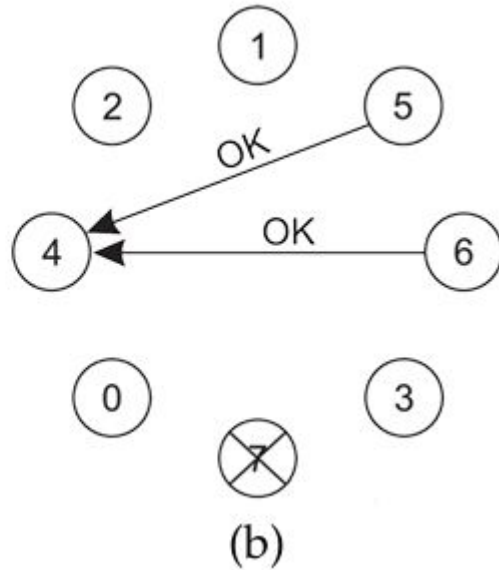
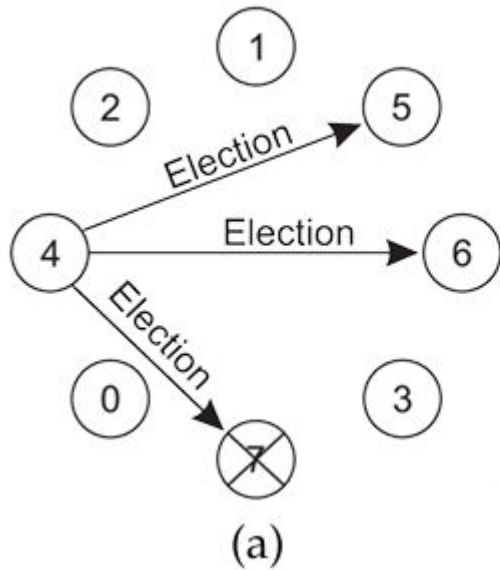
- O vencedor envia uma mensagem a todos os outros processos, informando que, a partir daquele momento, ele é o novo coordenador.

ALGORITMO Bully

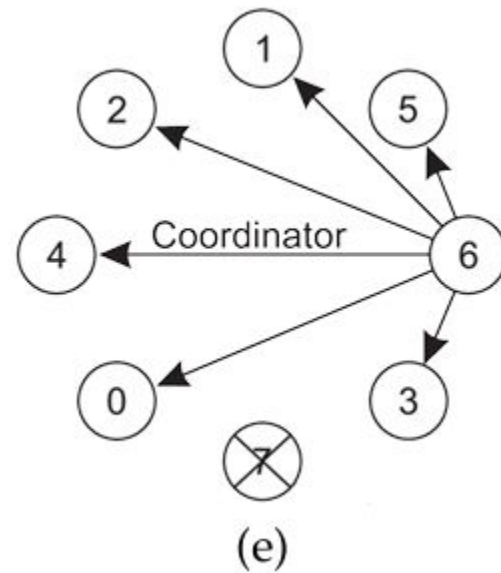
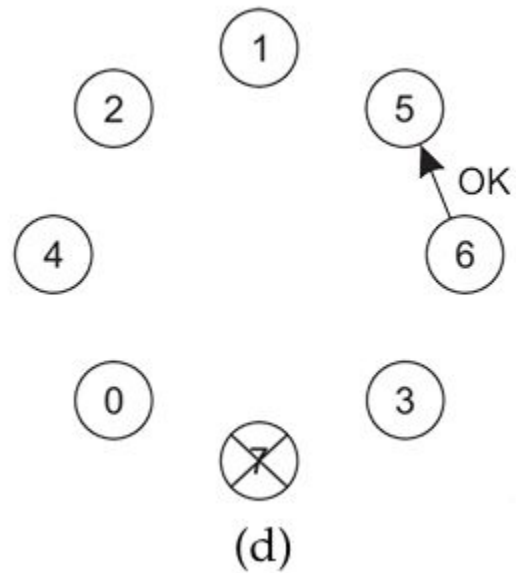
Se um processo que estava inativo retornar ao sistema, ele também inicia uma eleição. Caso seja o processo com o maior identificador entre os que estão em execução, ele vence a eleição e assume o papel de coordenador.

- processo com o maior identificador — o “mais poderoso” — sempre vence, o que dá origem ao nome “algoritmo do bully” (ou “algoritmo do valentão”).

ALGORITMO Bully

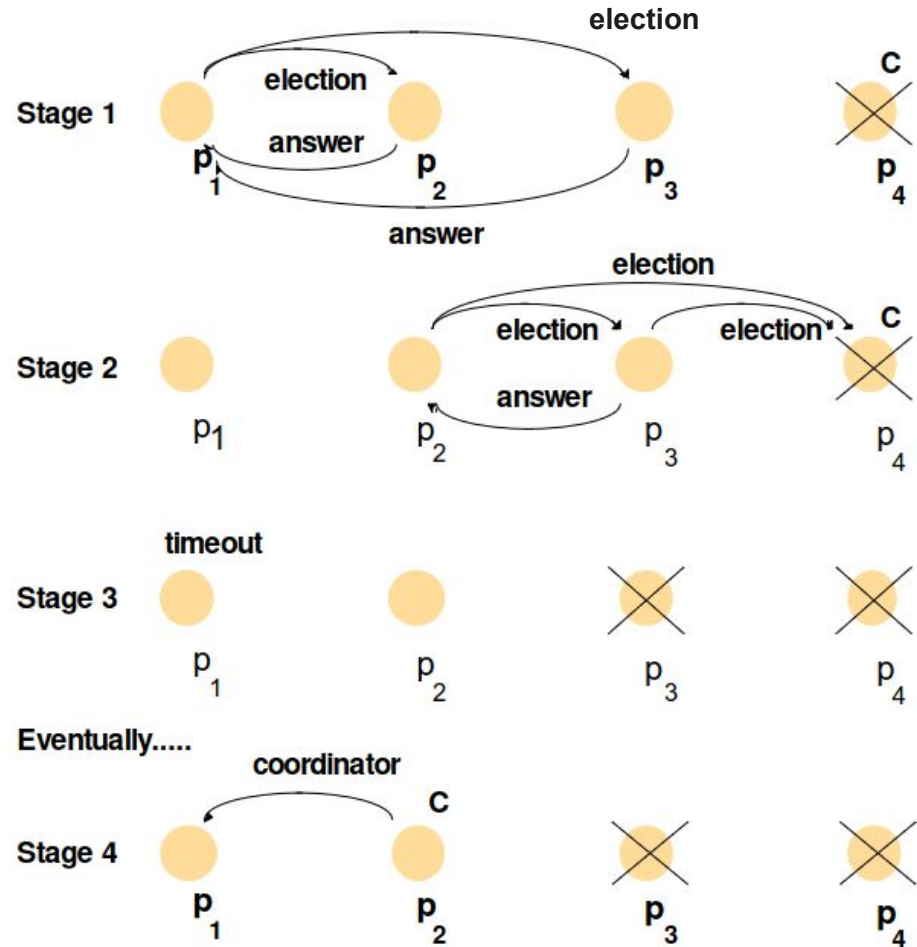


ALGORITMO Bully

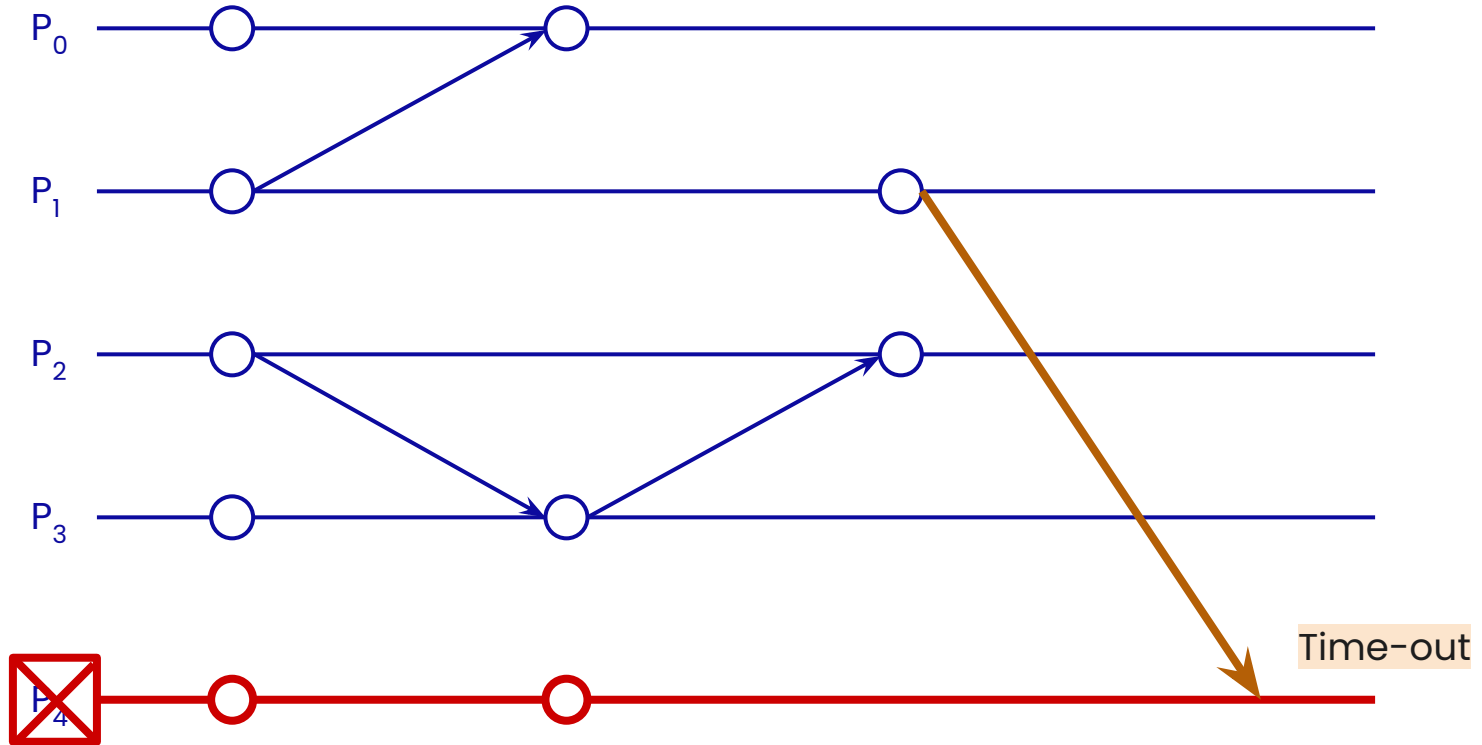


ALGORITMO Bully

A eleição do coordenador p_2 ,
após a falha de p_4 e depois de p_3



ALGORITMO Bully



ALGORITMO Ring (em anel)

O algoritmo de eleição baseado em anel lógico não utiliza um token como no algoritmo de exclusão mútua.

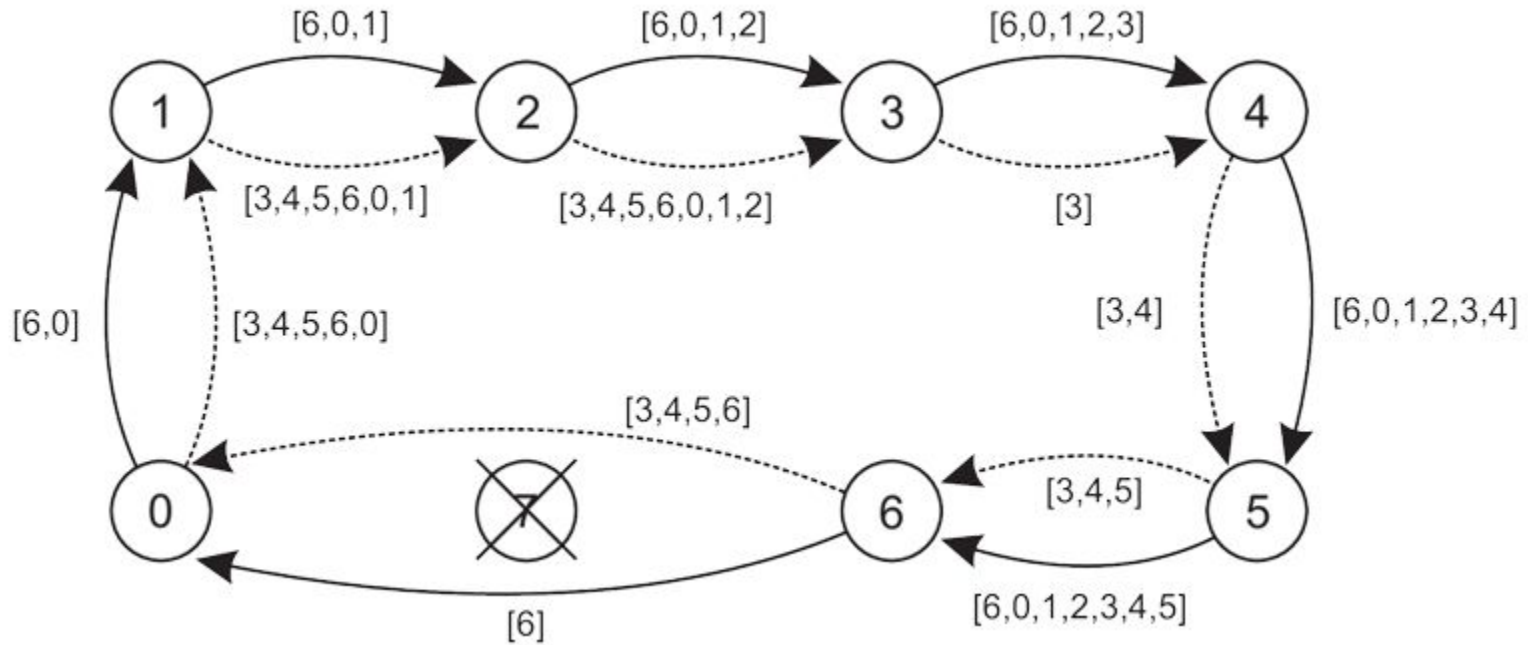
Assume-se que os processos estão organizados em uma ordem lógica circular e que cada processo conhece a configuração do anel, ou pelo menos possui informação suficiente para localizar o próximo processo ativo.

- Quando um processo detecta que o coordenador atual falhou, ele cria uma mensagem do tipo **ELECTION**, contendo seu próprio identificador, e tenta enviá-la ao seu sucessor lógico.
- Se esse sucessor não responder, o processo tenta o próximo processo na ordem do anel, repetindo esse procedimento até encontrar um processo ativo. A mensagem então continua circulando pelo anel até retornar ao processo que iniciou a eleição.

ALGORITMO Ring

- O processo que recebe uma mensagem **ELECTION** adiciona seu próprio identificador à lista contida na mensagem, indicando que está ativo e participando da eleição. Em seguida, encaminha a mensagem ao próximo processo ativo no anel.
- Eventualmente, a mensagem retorna ao processo que iniciou a eleição.
- Depois disso, o tipo da mensagem é alterado para **COORDINATOR**, e ela é enviada novamente ao longo do anel.
- Quando a mensagem **COORDINATOR** retorna ao processo que iniciou a eleição, ela é removida do sistema. A partir desse momento, todos os processos conhecem o novo coordenador e retornam ao funcionamento normal.

ALGORITMO Ring



ALGORITMO Ring

ANÁLISE

- Algoritmo Bully
 - Pior caso: processo com prioridade mais baixa detecta a falha do coordenador
 - $n-1$ eleições acontecem;
 - Quantidade de eleições conforme número de processos - $O(n^2)$ mensagens
 - Melhor caso: processo que detecta falha é o ativo de maior prioridade
 - $n - 2$ mensagens

ALGORITMO Ring

ANÁLISE

- Algoritmo Ring
 - eleição sempre precisa de $2n - 2$ mensagens;
 - $n - 1$ mensagens para rotação da mensagem de eleição;
 - $n - 1$ mensagens para rotação da mensagem de coordenador.

ALGORITMO Bully x Ring

Critério	Bully	Ring
Organização	Todos conhecem todos	Processos em anel lógico
Comunicação	Direta com processos de maior prioridade	Com o sucessor no anel
Líder eleito	Maior identificador ativo	Maior identificador ativo
Custo de mensagens	Pode chegar a $O(n^2)$	Geralmente $O(n)$
Melhor para	Sistemas pequenos/médios e totalmente conectados	Sistemas organizados em topologia lógica

ELEIÇÃO EM SISTEMAS DE GRANDE ESCALA

Muitos algoritmos de eleição de líder foram concebidos para sistemas distribuídos relativamente pequenos.

- Nós trocam informações em várias rodadas até chegarem coletivamente a um novo líder.

O problema fica bem mais complicado quando há muitos processos potenciais para escolher.

ELEIÇÃO EM AMBIENTES SEM FIO

Os algoritmos tradicionais de eleição de líder foram desenvolvidos considerando ambientes estáveis e considerando que:

- a troca de mensagens é confiável; e
- a topologia da rede não muda.

Na prática não se aplicam à maioria dos ambientes sem fio, especialmente quando há dispositivos móveis, onde a comunicação pode falhar e os nós podem entrar e sair da rede constantemente.

ELEIÇÃO EM AMBIENTES SEM FIO

Poucos protocolos de eleição funcionam bem em redes móveis

- Um exemplo é o protocolo de Vasudevan et al. (2004), que consegue lidar com falhas de nós e partições temporárias da rede.
 - permite eleger o melhor líder possível, em vez de simplesmente escolher um nó aleatoriamente.

ELEIÇÃO EM AMBIENTES SEM FIO

Funcionamento do protocolo de Vasudevan

- Vamos considerar apenas **redes ad hoc sem fio** (ou seja, sem infraestrutura fixa) e assumir que os nós não estão se movendo — como em uma rede de sensores, por exemplo.
1. **Início da eleição**: qualquer nó da rede pode iniciar uma eleição.
 - o nó envia uma mensagem ELECTION para todos os seus vizinhos imediatos (nós dentro do alcance de comunicação).

ELEIÇÃO EM AMBIENTES SEM FIO

Funcionamento do protocolo de Vasudevan

- Vamos considerar apenas **redes ad hoc sem fio** (ou seja, sem infraestrutura fixa) e assumir que os nós não estão se movendo — como em uma rede de sensores, por exemplo.

2. Propagação da eleição:

- a. Se um nó recebe uma mensagem **ELECTION** pela primeira vez:
 - define o remetente como seu **"pai"**; e
 - envia a mensagem **ELECTION** para todos os outros vizinhos, exceto o pai.

ELEIÇÃO EM AMBIENTES SEM FIO

Funcionamento do protocolo de Vasudevan

- Vamos considerar apenas **redes ad hoc sem fio** (ou seja, sem infraestrutura fixa) e assumir que os nós não estão se movendo — como em uma rede de sensores, por exemplo.
2. **Propagação da eleição:**
 - b. Se o nó receber uma mensagem ELECTION de outro vizinho:
 - apenas envia uma confirmação (ACK), sem repassar a mensagem.

ELEIÇÃO EM AMBIENTES SEM FIO

Funcionamento do protocolo de Vasudevan

- Vamos considerar apenas **redes ad hoc sem fio** (ou seja, sem infraestrutura fixa) e assumir que os nós não estão se movendo — como em uma rede de sensores, por exemplo.
3. **Confirmações e coleta de informações:**
 - a. Suponha que o nó **R** tenha definido o nó **Q** como pai.
 - **R** envia a mensagem ELECTION aos seus vizinhos (menos Q) e aguarda as confirmações (ACKs) de todos antes de responder ao pai **Q**.

ELEIÇÃO EM AMBIENTES SEM FIO

Funcionamento do protocolo de Vasudevan

- Vamos considerar apenas **redes ad hoc sem fio** (ou seja, sem infraestrutura fixa) e assumir que os nós não estão se movendo — como em uma rede de sensores, por exemplo.

3. Confirmações e coleta de informações:

b. Esse tempo de espera é importante:

- vizinhos que já escolheram um pai respondem rapidamente.
- Se todos os vizinhos possuírem pai, **R** é considerado um **nó folha** (ou seja, está no final de um ramo da rede).
- **R** pode enviar ao pai **Q** informações como tempo de bateria, capacidade de processamento e outros recursos

ELEIÇÃO EM AMBIENTES SEM FIO

Funcionamento do protocolo de Vasudevan

- Vamos considerar apenas **redes ad hoc sem fio** (ou seja, sem infraestrutura fixa) e assumir que os nós não estão se movendo — como em uma rede de sensores, por exemplo.

4. Escolha do melhor candidato:

- a. O nó **Q**, ao receber respostas de seus filhos, compara as informações (bateria, recursos, etc.) e escolhe o nó mais qualificado entre eles como candidato a líder.
- b. Em seguida, **Q** envia sua resposta ao pai **P**, repassando o candidato mais forte que encontrou em seu subgrupo.

ELEIÇÃO EM AMBIENTES SEM FIO

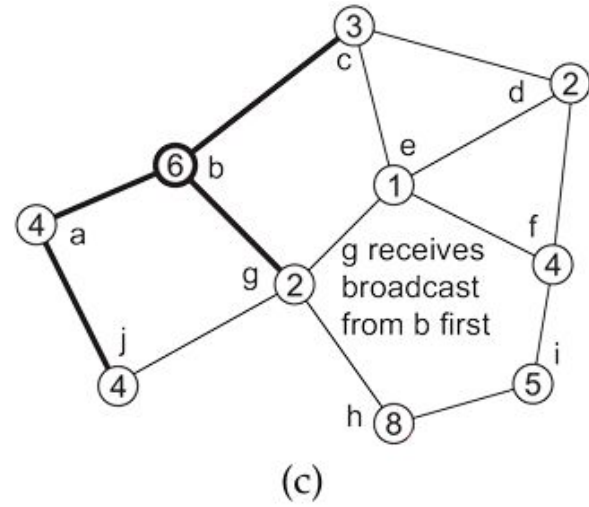
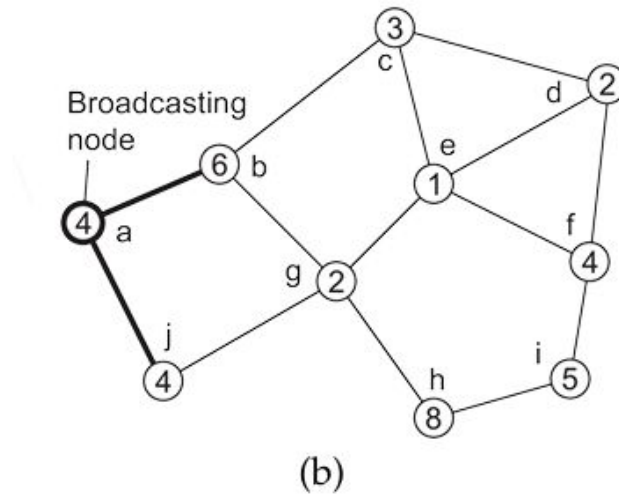
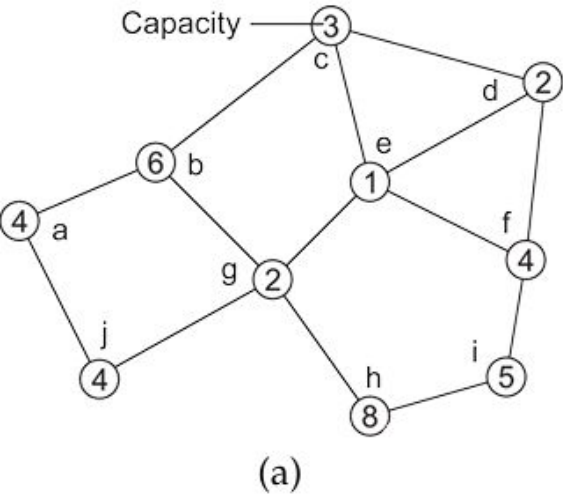
Funcionamento do protocolo de Vasudevan

- Vamos considerar apenas **redes ad hoc sem fio** (ou seja, sem infraestrutura fixa) e assumir que os nós não estão se movendo — como em uma rede de sensores, por exemplo.

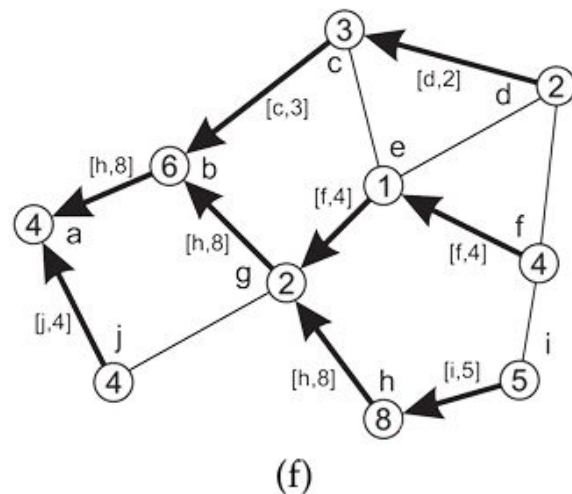
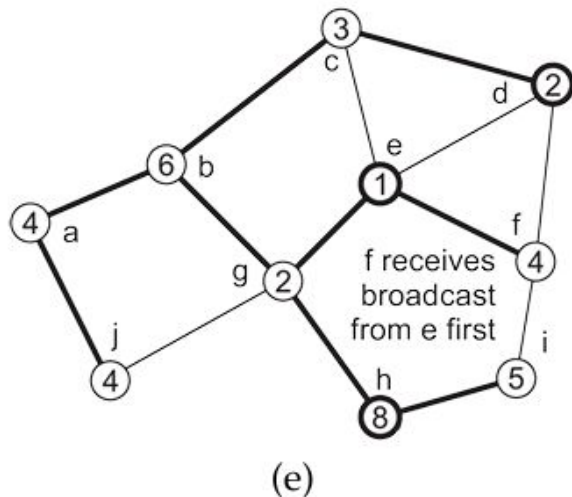
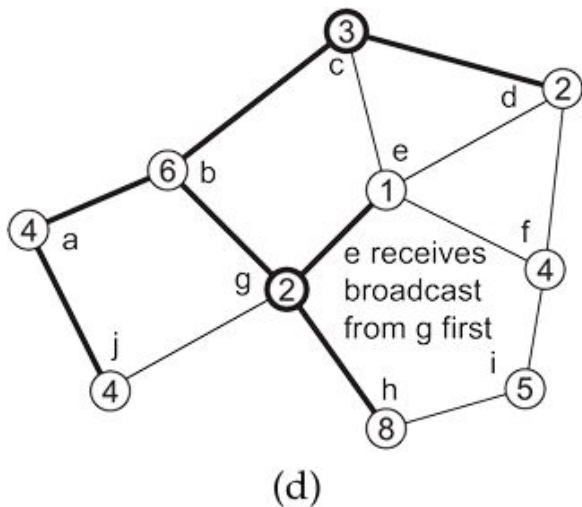
5. **Decisão final:**

- a. Esse processo se repete recursivamente, subindo pela “árvore” de mensagens até que a fonte original (o nó que iniciou a eleição) receba as respostas de todos.
- b. Com base nas informações, o nó fonte identifica o melhor candidato para ser o novo líder e anuncia o resultado para todos os nós da rede.

ELEIÇÃO EM AMBIENTES SEM FIO



ELEIÇÃO EM AMBIENTES SEM FIO



ELEIÇÃO EM AMBIENTES SEM FIO

No final, o nó fonte tem uma visão geral da rede e pode escolher o líder com mais recursos disponíveis (por exemplo, maior bateria, melhor desempenho, etc.).

- Mesmo em redes sem fio instáveis ou parcialmente desconectadas, o sistema consegue eleger um líder de forma adaptável.